1

BÖLÜM: KONTEYNER DÜNYASI & DOCKER TEMELLERI | MODULE: CONTAINER WORLD & DOCKER FUNDAMENTALS

"Ama Benim Bilgisayarımda Çalışıyordu!" – Sanal Makineler vs Konteynerler

"It Worked on My Machine!" – Virtual Machines vs Containers

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Eskiden bir tüccar kumaş, diğeri zeytinyağı fıçısı, öbürü gevşek buğday çuvalı taşırdı. Gemiyi yüklemek günler sürer, çuvallar patlar, her şey birbirine karışırdı! 1956'da standart çelik kargo konteyneri icat edildi. Artık gemi vincinin içinde ne olduğunu bilmesine gerek yok; standart kutuyu alır, koyar! Docker da yazılım dünyasının standart çelik kutusudur. Sanal Makine (VM) koca bir evi temeliyle taşımak gibidir; Docker Konteyneri ise sadece eşyalarınızı bavula koyup taşımak gibidir!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Back in the 1950s, dockers loaded odd wooden barrels, loose bags of grain, and fragile crates by hand. Cargo took days to load and frequently broke. In 1956, the standardized steel shipping container changed human trade forever! Cranes didn't care what was inside—only that it fit the standard locks. Docker is that exact shipping container for software. A Virtual Machine (VM) is like transporting an entire house including foundations; a Container is packing just your suitcase with only the essentials!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Sanal Makineler (Hypervisor Type 1/2) her uygulama için ayrı bir misafir işletim sistemi (Guest OS), ayrı sanal donanım ve kendi çekirdeğini (Kernel) ayağa kaldırır. Bu da gigabaytlarca RAM ve dakikalar süren açılış süresi demektir. Docker konteynerleri ise doğrudan ana makinenin Linux çekirdeğini paylaşır. İzolasyon iki temel Linux çekirdek özelliğiyle sağlanır: 1) Namespaces (Konteynerin sadece kendi süreçlerini, ağını ve dosya sistemini görmesini sağlar; sanal göz bağıdır), 2) Cgroups (Control Groups: CPU ve RAM kotasını sınırlar).

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Virtual Machines (Type 1/2 Hypervisors) virtualize physical hardware. Every VM runs a complete Guest OS kernel, consuming gigabytes of memory and taking minutes to boot. Docker containers share the host Linux kernel. Isolation is achieved via two core Linux kernel primitives: 1) Namespaces (isolate PID, Mount, Net, IPC, UTS, User so processes think they own the machine), and 2) Cgroups (Control Groups, metering and limiting CPU, RAM, and IOPS).

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# Doğrudan sunucuda çalıştırmak (Kötü Yöntem):
# Java 17 mi kurulu, 21 mi? Hangi glibc sürümü var? Kimse bilmez!
java -jar target/spring-boot-k8s-eli5-1.0.0.jar

# Docker ile standart çelik konteynerde çalıştırmak (Modern Yöntem):
docker run -d -p 8080:8080 --name my-spring-coffee ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Konteynerler tam anlamıyla bir sanal makine değildir! Ana işletim sisteminin çekirdeğini (kernel) kullandıkları için, Linux tabanlı bir konteyner doğrudan saf Windows çekirdeğinde çalışmaz (arka planda WSL2 veya Hyper-V Linux VM gerekir).

🇬🇧 WATCH OUT

Containers are NOT lightweight virtual machines! Because they share the host kernel, Linux containers on Windows/macOS secretly run inside a thin Linux VM (WSL2 / Apple Hypervisor).

🎯 Özet Çıkarımlar

- Konteynerler Guest OS yükünü ortadan kaldırarak saniyeler içinde ayağa kalkar.
- Linux Namespaces görünürlüğü, Cgroups ise CPU/Bellek limitlerini kontrol eder.
- Java uygulaması her işletim sisteminde aynı JRE ve bağımlılıklarla garanti çalışır.

🎯 Key Takeaways

- Containers eliminate Guest OS overhead, starting in milliseconds.
- Namespaces provide visibility boundaries; Cgroups enforce resource limits.
- Guarantees identical Java runtime behavior across laptops, staging, and production.